Reviewer Pack — Minimal Rank of Noncrossing Partitions vs Communication Comp…

Entry cdb740b38461 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 05:10:33 UTC

Minimal Rank of Noncrossing Partitions vs Communication Complexity for Sorting

Entry ID: `cdb740b38461` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 05:10:33 UTC

1. Conjecture

Field A (mathematical branch): Enumerative Combinatorics (Noncrossing Partitions) **Field B** (complexity object): Communication Complexity: Sorting

Statement:

> ['For any instance of the sorting problem with n elements, the minimal rank of the associated noncrossing partition is upper-bounded by O(log n). Specifically, there exists a constant c such that for all instances I with |I| ≤ 40, the minimal rank of the noncrossing partition π(I) is at most c * log(|I|).', 'Additionally, for any instance I with |I| ≤ 40 and a communication protocol P used to sort I, if P requires at least k bits of communication, then there exists a noncrossing partition π(I) such that the rank of π(I) is at most k.', 'The minimal rank of a noncrossing partition is defined as the length of the longest chain in the corresponding Young diagram.']

Rationale (proposer's reasoning):

> ['Noncrossing partitions encode combinatorial structures that are closely related to sorting algorithms. They can be used to model the state of a comparison-based sorting algorithm, and their ranks might correspond to the depth of recursive calls or the number of comparisons made during the sorting process.', 'This conjecture posits that the complexity of communication protocols for sorting is bounded by the combinatorial structure of noncrossing partitions. If true, it would provide a new perspective on the relationship between enumerative combinatorics and complexity theory.']

Taxonomy category: `ENUMERATIVE_COMBINATORICS` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d4036f7cbe030d02`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For an instance I with |I| ≤ 40, if the minimal rank of the associated noncrossing partition π(I) is at most c * log(|I|) and there exists a communication protocol P that requires at least k bits for sorting I such that π(I) has a rank ≤ k.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal Rank Noncrossing Partition AND Communication Complexity Sorting` - `Young Diagram Length O(log n) IN Enumerative Combinatorics AND Sorting Algorithm` - `Noncrossing Partition Rank Bound Communication Protocol`

Top relevant hits considered: - [http://arxiv.org/abs/1509.06942v2] Symmetric Decompositions and the Strong Sperner Property for Noncrossing Partition Lattices - [http://arxiv.org/abs/1604.06009v2] Oriented Flip Graphs and Noncrossing Tree Partitions - [http://arxiv.org/abs/2212.13799v6] Noncrossing partitions of a marked surface - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \to \mu^+\mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_random_permutation(n):
    return list(range(1, n + 1))

def noncrossing_partition_rank(permutation):
    n = len(permutation)
    if n == 0:
        return 0

    # Construct the Young diagram
    young_diagram = []
    for i in range(n):
        row = [permutation[i]]
        j = permutation[i]
```

```
        while j != i + 1:
            j = permutation[j - 1]
            row.append(j)
        young_diagram.append(row)

    # Compute the rank of the Young diagram
    rank = 0
    for row in young_diagram:
        if len(row) > rank:
            rank = len(row)

    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        instances_tested = 0
        total_rank = 0
        total_communication_bits = 0

        for _ in range(30):
            permutation = generate_random_permutation(n)
            rank = noncrossing_partition_rank(permutation)
            communication_bits = random.randint(1, n * math.log2(n))  # Simulate communication bit

            instances_tested += 1
            total_rank += rank
            total_communication_bits += communication_bits

        mean_rank = total_rank / instances_tested
        mean_communication_bits = total_communication_bits / instances_tested

        if mean_rank > mean_communication_bits:
            conjecture_holds = False
            counterexample = f"n={n}, mean_rank={mean_rank}, mean_communication_bits={mean_communi
        else:
            conjecture_holds = True
            counterexample = ""

        results.append({
            "metric_name": "communication_complexity",
            "metric_value": mean_communication_bits,
            "instances_tested": instances_tested,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        })

    return {
        "seed": seed,
        "results": results
    }

if __name__ == "__main__":
```

```python
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.extend(result["results"])

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank) ** 2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fracti
    elif any(r["counterexample"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if r["conjecture_holds"] is False)
        counterexample_desc = next(r["counterexample"] for r in results if r["conjecture_holds"] i
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample_desc}\" first_failing_seed={fir
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_45d283ad.py", line 92, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_45d283ad.py", line 57, in run_trial
    communication_bits = random.randint(1, n * math.log2(n))  # Simulate communication bits
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 312, in randrange
    istop = _index(stop)
            ^^^^^^^^^^^^
TypeError: 'float' object cannot be interpreted as an integer
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18452
2	preregistration	ollama_remote	glm4:latest	0	0	9740
3	novelty	ollama_remote	glm4:latest	0	0	8399
4	novelty	ollama_remote	glm4:latest	0	0	8997
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14169
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8808
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7938
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9582
9	judge	ollama_remote	glm4:latest	0	0	13047

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 99132 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/cdb740b38461.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cdb740b38461.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/cdb740b38461.tar.gz` (if generated)